Hashing และการใช้แฮชด้วย STL

โน้ตนี้อธิบายเรื่อง Hashing และ STL tools ที่ใช้ในการแข่งขัน โดยเป้าหมายคือให้นักเรียนเข้าใจว่าแฮชแก้ปัญหาอะไร ใช้เมื่อไหร่ และใช้ STL ได้ถูกต้องและมีประสิทธิภาพ

---

1. Hashing คืออะไร?

Hashing คือเทคนิคที่แปลง **key** ให้เป็นตัวเลข (hash value) เพื่อกำหนดว่าข้อมูลจะถูกเก็บที่ไหน
เป้าหมาย:

- ค้นหาข้อมูลได้เร็ว
- เพิ่มข้อมูลได้เร็ว
- เช็คว่ามีข้อมูลอยู่ไหมได้เร็ว

ทำให้ operations ทั่วไปเป็น O(1) เฉลี่ย แทนที่จะเป็น O(N) จาก linear scan
ตัวอย่างปัญหาที่ hashing แก้ได้อย่างมีประสิทธิภาพ:

- เคยเห็นตัวเลขนี้มาก่อนไหม?
- ค่านี้ปรากฏกี่ครั้ง?
- key นี้ map ไปที่ value อะไร?

---

2. โครงสร้างของ Hash Table

Hash table ทำงานใน 3 ขั้นตอน:

1. แปลง **key** เป็นตัวเลขด้วย **hash function**
2. ใช้ตัวเลขนั้นกำหนด **bucket (ช่องเก็บ)**
3. เก็บ **ข้อมูล** ไว้ใน bucket นั้น

```
Key → Hash Function → Bucket Index → เก็บข้อมูล

ตัวอย่าง:
"apple" → hash("apple") → 17 → bucket[17] ← เก็บ "apple"

ค้นหาทีหลัง:
"apple" → hash("apple") → 17 → เปิด bucket[17] → พบ
```

ผลต่อ Time Complexity:
ถ้า hash function กระจาย key ได้ดี แต่ละ bucket มี element น้อย → lookup เฉลี่ย O(1)
Worst case (ทุก key ไป bucket เดียวกัน) = O(N) แต่เกิดขึ้นยากมากถ้าใช้ hash function ที่ดี

---

3. Hash Function

คุณสมบัติของ hash function ที่ดี:

คุณสมบัติ	ความหมาย
คำนวณเร็ว	ใช้เวลาน้อย
กระจายสม่ำเสมอ	ทุก bucket ได้ข้อมูลใกล้เคียงกัน
collision น้อย	key ต่างกัน → bucket ต่างกัน

```
hash function แย่:  hash(x) = 0        ← ทุกอย่างไป bucket เดียว → O(N)
hash function ดี:   hash(x) = x mod p  ← p เป็น prime ขนาดใหญ่
```

3.1 Hash Function สำหรับ Integer

วิธีพื้นฐานที่สุด: hash(x) = x mod BUCKETS
โค้ดด้านล่างแสดงการกระจาย key ลง bucket จริง — สังเกตว่าแต่ละ bucket ได้รับ key กี่ตัว

In [ ]:

```c
#include <stdio.h>

#define BUCKETS  10
#define NUM_KEYS 20

int main() {
    int keys[NUM_KEYS] = {23, 57, 31, 14, 99, 45, 13, 33, 87, 62,
                          10, 77, 43, 26, 95, 38, 51, 72, 19, 84};
    int bucket_count[BUCKETS] = {0};

    printf("Key -> Bucket (hash = key mod 10)\n");
    printf("------------------------------\n");

    for (int i = 0; i < NUM_KEYS; i++) {
        int k = keys[i];
        int b = k % BUCKETS;
        bucket_count[b]++;
        printf("key=%3d -> bucket[%d]\n", k, b);
    }

    printf("\nสรุปจำนวน key ในแต่ละ bucket:\n");
    for (int i = 0; i < BUCKETS; i++)
        printf("bucket[%d] = %d keys\n", i, bucket_count[i]);

    return 0;
}
```

3.2 Hash Collision คืออะไร?

Collision คือสถานการณ์ที่ key ต่างกันได้ hash value เหมือนกัน

```
hash(x) = x mod 10

23 → bucket[3]
13 → bucket[3]   ← collision!
33 → bucket[3]   ← collision!
```

Collision ไม่สามารถหลีกเลี่ยงได้สมบูรณ์ (เพราะ key มีได้ไม่จำกัด แต่ bucket มีจำนวนจำกัด)
สิ่งที่ทำได้คือ **ลดให้น้อยที่สุด** และ **จัดการอย่างมีประสิทธิภาพ**

3.3 วิธีจัดการ Collision — Separate Chaining

วิธีที่ 1: **Separate Chaining** — แต่ละ bucket เก็บ linked list ของ key ที่ชนกัน

```
bucket[3]: 23 → 13 → 33 → null
bucket[7]: 57 → null
bucket[1]: 31 → null
```

ขั้นตอนค้นหา 13 :

1. คำนวณ 13 mod 10 = 3
2. เปิด bucket[3]
3. เดิน list: 23 → 13 ✓ พบ

โค้ดด้านล่างจำลอง separate chaining ด้วย memory pool + linked list

In [ ]:

```c
#include <stdio.h>

#define NUM_BUCKETS 10
#define MAX_NODES   50

struct Node {
    int key;
    int next;  // index ของ node ถัดไปใน pool (-1 = ไม่มี)
};

struct Node pool[MAX_NODES];   // memory pool เก็บ node ทั้งหมด
int pool_top = 0;              // ตำแหน่งถัดไปที่ว่างใน pool

int bucket[NUM_BUCKETS];       // bucket[i] = index node แรกใน bucket i

int hash_fn(int key) {
    return ((key % NUM_BUCKETS) + NUM_BUCKETS) % NUM_BUCKETS;  // ป้องกันค่า key เป็นค่าลบ
}

void init() {
    for (int i = 0; i < NUM_BUCKETS; i++) bucket[i] = -1;
}

void insert(int key) {
    int b = hash_fn(key);           // คำนวณ bucket ที่จะเก็บ

    pool[pool_top].key  = key;
    pool[pool_top].next = bucket[b]; // เชื่อมกับ node เดิมใน bucket
    bucket[b] = pool_top++;          // bucket ชี้มาที่ node ใหม่
}

int find_key(int key) {
    int b = hash_fn(key);            // หา bucket ที่ key ควรอยู่

    for (int i = bucket[b]; i != -1; i = pool[i].next)
        if (pool[i].key == key) return 1;
    return 0;
}

void print_table() {
    for (int b = 0; b < NUM_BUCKETS; b++) {
        int count = 0;
        printf("bucket[%d]: ", b);
        for (int i = bucket[b]; i != -1; i = pool[i].next) {
            printf("%d ", pool[i].key);
            count++;
        }
        if (count > 1) printf(" <- collision! (%d keys)", count);
        printf("\n");
    }
}

int main() {
    init();

    int keys[] = {23, 13, 33, 57, 31, 99, 43, 87, 3, 53};
    int n = sizeof(keys) / sizeof(keys[0]);
    for (int i = 0; i < n; i++) insert(keys[i]);

    printf("Separate Chaining — สถานะหลัง insert:\n");
    print_table();

    printf("\nการค้นหา:\n");
    printf("find(13) = %s\n", find_key(13) ? "พบ" : "ไม่พบ");
    printf("find(99) = %s\n", find_key(99) ? "พบ" : "ไม่พบ");
    printf("find(50) = %s\n", find_key(50) ? "พบ" : "ไม่พบ");

```

```
    return 0;
}
```

3.4 วิธีจัดการ Collision — Linear Probing

วิธีที่ 2: **Linear Probing** — ถ้า bucket เต็มให้เลื่อนไป bucket ถัดไปจนเจอที่ว่าง

```
insert(23): hash=3 → bucket[3] ว่าง → เก็บที่ 3
insert(13): hash=3 → bucket[3] เต็ม → ลองที่ 4 ว่าง → เก็บที่ 4
insert(33): hash=3 → 3 เต็ม → 4 เต็ม → 5 ว่าง → เก็บที่ 5
```

เปรียบเทียบสองวิธี:

	Separate Chaining	Linear Probing
โครงสร้าง	array + linked list	array เดียว
memory	มากกว่า (pointer)	น้อยกว่า
cache friendly	❌	✅
ลบข้อมูล	ง่าย	ต้องระวัง
STL ใช้แบบไหน?	chaining (unordered_map)	—

โค้ดด้านล่างสาธิต linear probing

In [ ]:

```c
#include <stdio.h>

#define SZ    11   // prime size ช่วยกระจาย
#define EMPTY -1

int table[SZ];

void init() {
    for (int i = 0; i < SZ; i++) table[i] = EMPTY;
}

int hash_fn(int key) {
    return ((key % SZ) + SZ) % SZ;
}

int insert(int key) {
    int h = hash_fn(key);
    for (int i = 0; i < SZ; i++) {
        int pos = (h + i) % SZ;
        if (table[pos] == EMPTY || table[pos] == key) {
            table[pos] = key;
            return 1;
        }
    }
    return 0;  // table เต็ม
}

int find_key(int key) {
    int h = hash_fn(key);
    for (int i = 0; i < SZ; i++) {
        int pos = (h + i) % SZ;
        if (table[pos] == EMPTY) return 0;
        if (table[pos] == key)   return 1;
    }
    return 0;
}

void print_table() {
    for (int i = 0; i < SZ; i++) {
        if (table[i] == EMPTY) printf("[%2d] ---\n", i);
        else                   printf("[%2d] %d\n", i, table[i]);
    }
}

int main() {
    init();

    int keys[] = {10, 22, 31, 4, 15, 28, 17, 88, 59};
    int n = sizeof(keys) / sizeof(keys[0]);
    for (int i = 0; i < n; i++) insert(keys[i]);

    printf("Linear Probing (size=%d):\n", SZ);
    print_table();

    printf("\nการค้นหา:\n");
    printf("find(15) = %s\n", find_key(15) ? "พบ" : "ไม่พบ");
    printf("find(99) = %s\n", find_key(99) ? "พบ" : "ไม่พบ");

    return 0;
}
```

3.5 Hash Function สำหรับ String

String ใช้ modulo โดยตรงไม่ได้ ต้องแปลง character เป็นตัวเลขก่อน

วิธีที่ 1 — Sum of ASCII (ง่ายแต่มีจุดอ่อน):

```
hash(s) = (ผลรวม ASCII ของทุก character) mod table_size
```

จุดอ่อน: anagram ได้ hash เหมือนกัน — "abc" และ "bca" มี ASCII sum เท่ากัน
วิธีที่ 2 — Polynomial Hash (แนะนำ):

$$hash(s) = (s[0]\times p^0 + s[1]\times p^1 + s[2]\times p^2 + \ldots) \bmod m$$

- p = base นิยมใช้ 31 หรือ 53
  - ใช้ lowercase เท่านั้น → base = 31 (prime > 26)
  - ใช้ทั้ง lower+upper → base = 53 (prime > 52)
- m = ขนาด table

การคูณด้วย power ทำให้ตำแหน่งของ character มีผลต่อ hash ทำให้ anagram ได้ค่าต่างกัน

Modular arithmetic มีสมบัติ:

$$(a + b) \bmod m = ((a \bmod m) + (b \bmod m)) \bmod m$$
$$(a \times b) \bmod m = ((a \bmod m) \times (b \bmod m)) \bmod m$$

In [ ]:

```c
#include <stdio.h>

int hash_sum(const char *s, int m) {
    int h = 0;
    for (int i = 0; s[i]; i++) h += (int)s[i];
    return h % m;
}

int hash_poly(const char *s, int m) {
    long long h = 0, p = 1;
    const int BASE = 31;
    for (int i = 0; s[i]; i++) {
        h = (h + (s[i] - 'a' + 1) * p) % m;
        p = (p * BASE) % m;
    }
    return (int)h;
}

int main() {
    const int M = 100;

    // anagram — ตัวอักษรเหมือนกัน เรียงต่างกัน
    const char *anagrams[] = {"abc", "bca", "cab", "acb", "bac", "cba"};
    int n_anagram = 6;

    printf("=== จุดอ่อนของ sum hash: anagram ได้ hash เดียวกัน ===\n");
    for (int i = 0; i < n_anagram; i++)
        printf("  hash_sum(\"%s\") = %d\n", anagrams[i], hash_sum(anagrams[i], M));

    printf("\n=== Polynomial hash แก้ได้: แต่ละตัวได้ค่าต่างกัน ===\n");
    for (int i = 0; i < n_anagram; i++)
        printf("  hash_poly(\"%s\") = %d\n", anagrams[i], hash_poly(anagrams[i], M));

    return 0;
}
```

4. จาก Manual Hash สู่ STL

จาก Section 3 เราเขียน hash table เองทั้งหมด ซึ่งต้องดูแลหลายส่วน:

สิ่งที่ต้องทำเอง (manual)	STL จัดการให้
เขียน hash function	✅ built-in สำหรับ type พื้นฐาน
จอง array ของ bucket	✅ อัตโนมัติ
จัดการ collision (chaining)	✅ อัตโนมัติ
resize เมื่อ table เต็ม (rehashing)	✅ อัตโนมัติ

ตั้งแต่ Section 5 เป็นต้นไปเราจะใช้ STL แทน — โค้ดสั้นลงมาก และปลอดภัยกว่า

5. STL สำหรับ Hashing

โครงสร้าง	จุดประสงค์	key ซ้ำ?
unordered_set	เก็บ key ที่ไม่ซ้ำ	❌
unordered_map	เก็บ key → value	❌
unordered_multiset	เก็บ key ซ้ำได้	✅
unordered_multimap	เก็บ key ซ้ำได้พร้อม value	✅

ทุกโครงสร้างใช้ **hash table ภายใน** — ใช้บ่อยที่สุดคือ unordered_set และ unordered_map

6. Hash vs Tree — เปรียบเทียบให้เห็นชัด

โครงสร้าง	โครงสร้างภายใน	insert/find	key เรียงลำดับ?
set	balanced BST	O(log N)	✅
map	balanced BST	O(log N)	✅
unordered_set	hash table	O(1) เฉลี่ย	❌
unordered_map	hash table	O(1) เฉลี่ย	❌

> **กฎเลือก:** ต้องการเร็วสูงสุด → unordered_*
> ต้องการลำดับ / lower_bound / upper_bound → set / map

7. Operations พื้นฐาน + ตัวอย่าง

7.1 unordered_set

Operation	ความหมาย	Complexity
insert(x)	เพิ่ม element	O(1) เฉลี่ย
erase(x)	ลบ element	O(1) เฉลี่ย
count(x)	เช็คว่ามีอยู่ไหม (0 หรือ 1)	O(1) เฉลี่ย
find(x)	คืน iterator (ใช้เช็คแบบ safe)	O(1) เฉลี่ย
size()	จำนวน element	O(1)

ตัวอย่าง: ตรวจจับตัวเลขซ้ำ

In [8]:

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    unordered_set<int> seen;
    vector<int> nums = {5, 3, 8, 1, 9, 3, 7, 2, 8, 4, 6, 1, 10};

    cout << "ลำดับ: ";
    for (int x : nums) cout << x << " ";
    cout << "\n\n";

    for (int x : nums) {
        if (seen.count(x)) {
            cout << "ตัวแรกที่ซ้ำ: " << x << "\n";
            return 0;
        }
        seen.insert(x);
    }
    cout << "ไม่มีตัวซ้ำ\n";
    return 0;
}
```

```
ลำดับ: 5 3 8 1 9 3 7 2 8 4 6 1 10

ตัวแรกที่ซ้ำ: 3
```

7.2 unordered_map

Operation	ความหมาย	หมายเหตุ
mp[key]	เข้าถึง value (สร้าง key ถ้าไม่มี)	⚠️ ดู Section 8
mp[key] = v	กำหนด value	–
find(key)	ค้นหา key – ไม่สร้าง key ใหม่	✅ แนะนำสำหรับเช็ค
count(key)	เช็คว่ามี key ไหม (0 หรือ 1)	✅ แนะนำสำหรับเช็ค
erase(key)	ลบ key	–

ตัวอย่าง: นับความถี่คำ

In [ ]:

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    unordered_map<string, int> freq;

    vector<string> words = {
        "apple", "banana", "apple", "cherry", "banana",
        "apple", "date", "cherry", "apple", "banana"
    };

    for (const string &s : words)
        freq[s]++;  // key ไม่มี → สร้างอัตโนมัติ ค่าเริ่มต้น = 0

    cout << "ความถี่แต่ละคำ:\n";
    for (auto [word, count] : freq)
        cout << "  " << word << ": " << count << "\n";

    // หาคำที่บ่อยที่สุด
    string best;
    int best_cnt = 0;
    for (auto [word, count] : freq)
        if (count > best_cnt) { best_cnt = count; best = word; }

    cout << "\nคำที่บ่อยที่สุด: \"" << best << "\" (" << best_cnt << " ครั้ง)\n";
    return 0;
}
```

---

8. ⚠️ จุดที่มักผิด: `operator[]` vs `find()`

นักเรียนมักเขียนแบบนี้เพื่อเช็คว่ามี key ไหม:

```cpp
if (mp[key] == 0)  // ❌ ผิด!
```

ปัญหา: `mp[key]` จะ **สร้าง key ทันที** ถ้ายังไม่มี — map จะโตขึ้นโดยไม่ตั้งใจ
โค้ดด้านล่างแสดงให้เห็น side effect นี้จริงๆ

In [ ]:

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    unordered_map<string, int> mp;
    mp["apple"] = 3;
    mp["banana"] = 1;

    cout << "map เริ่มต้นมี " << mp.size() << " keys\n";

    // ❌ แบบผิด: [] เช็ค → สร้าง key โดยไม่ตั้งใจ
    if (mp["cherry"] == 0) cout << "cherry ไม่มีอยู่ (แต่ถูกสร้างแล้ว!)\n";
    if (mp["date"]   == 0) cout << "date ไม่มีอยู่ (แต่ถูกสร้างแล้ว!)\n";

    cout << "หลังใช้ []: map มี " << mp.size() << " keys <- เพิ่มขึ้น!\n";
    for (auto [k, v] : mp) cout << "  " << k << "(" << v << ")\n";

    cout << "\n";

    // ✅ แบบถูก: find() / count() ไม่สร้าง key
    unordered_map<string, int> mp2;
    mp2["apple"] = 3;
    mp2["banana"] = 1;

    cout << "map2 เริ่มต้นมี " << mp2.size() << " keys\n";
    if (mp2.find("cherry") == mp2.end()) cout << "cherry ไม่มีอยู่\n";
    if (mp2.count("date") == 0)          cout << "date ไม่มีอยู่\n";
    cout << "หลังใช้ find(): map2 มี " << mp2.size() << " keys <- ไม่เปลี่ยน\n";

    return 0;
}
```

กฎง่ายๆ:

จุดประสงค์	วิธีที่ใช้
เช็คว่ามี key ไหม	find() หรือ count()
อ่านหรือเขียน value	mp[key]

---

9. Custom Hash สำหรับ Complex Keys

บางครั้ง key ไม่ใช่ type พื้นฐาน เช่น pair<int,int>
STL ไม่มี default hash ให้ ต้องกำหนดเอง

In [10]:

```cpp
#include <bits/stdc++.h>
using namespace std;

// แปลง (row, col) เป็น long long เพื่อใช้เป็น key ใน unordered_map
// row เก็บไว้ใน 32 bit สูง, col เก็บไว้ใน 32 bit ต่ำ
long long encode(int row, int col) {
    return ((long long)row << 32) | (unsigned int)col;
}

int main() {
    // key = encode(row, col), value = distance
    unordered_map<long long, int> dist;

    dist[encode(0, 0)] = 0;
    dist[encode(0, 1)] = 1;
    dist[encode(1, 0)] = 1;
    dist[encode(1, 1)] = 2;
    dist[encode(2, 3)] = 5;

    cout << "Distance map (row,col) -> dist:\n";
    for (auto [key, d] : dist) {
        int row = key >> 32;
        int col = key & 0xFFFFFFFF;
        cout << "  (" << row << ", " << col << ") -> " << d << "\n";
    }

    // ค้นหา — ใช้ find() ไม่ใช่ [] เพื่อไม่ให้สร้าง key ใหม่โดยไม่ตั้งใจ
    auto print_dist = [&](int row, int col) {  // ใช้ [&] เพราะต้องการเข้าถึงตัวแปร dist
        auto it = dist.find(encode(row, col));
        if (it != dist.end())
            cout << "dist(" << row << ", " << col << ") = " << it->second << "\n";
        else
            cout << "dist(" << row << ", " << col << ") = ไม่มี\n";
    };

    cout << "\n";
    print_dist(1, 1);  // พบ → 2
    print_dist(5, 5);  // ไม่มี

    return 0;
}
```

```
Distance map (row,col) -> dist:
(2, 3) -> 5
(1, 1) -> 2
(1, 0) -> 1
(0, 1) -> 1
(0, 0) -> 0

dist(1, 1) = 2
dist(5, 5) = ไม่มี
```

---

10. เทคนิคเพิ่มประสิทธิภาพ

ถ้าคาดว่าจะมี element จำนวนมาก ให้ **จอง bucket ล่วงหน้า** เพื่อหลีกเลี่ยง rehashing

```cpp
unordered_map<int, int> mp;
mp.reserve(200000);       // จอง bucket ไว้ล่วงหน้า
mp.max_load_factor(0.7);  // ลด collision
```

ทำไม rehashing ถึงช้า: เมื่อ element มากเกินขีด STL จะ

1. จอง array ใหม่ขนาดใหญ่กว่าเดิม 2 เท่า
2. ย้าย element ทั้งหมดไปคำนวณ hash ใหม่
3. ทิ้ง array เก่า

การ reserve ป้องกันขั้นตอนนี้ไม่ให้เกิดระหว่างโปรแกรมทำงาน

---

11. Benchmark — วัดเวลาจริง

เปรียบเทียบ Linear scan O(N) vs Binary search O(log N) vs Hash O(1)
ใช้ข้อมูล 5,000,000 รายการ

ข้อมูลทดสอบ — mt19937

โค้ดนี้ใช้**คลาส** mt19937 สร้างข้อมูลทดสอบแทนการรับจาก input

```
mt19937 rng(42);    // สร้าง random number generator seed=42
rng()               // เรียกแต่ละครั้งได้ตัวเลขถัดไปในลำดับ
rng() % (N * 2)    // จำกัดช่วงด้วย mod
```

seed 42 — ทำให้ได้ข้อมูลชุดเดิมทุกครั้งที่รัน เพื่อให้เปรียบเทียบเวลาได้ยุติธรรม
ถ้าเปลี่ยน seed จะได้ข้อมูลชุดต่างออกไป แต่ผลเปรียบเทียบ O(1) vs O(log N) vs O(N) ยังเหมือนเดิม

In [ ]:

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    const int N = 5000000;
    const int Q = 1000000;

    mt19937 rng(42);
    vector<int> data(N), queries(Q);
    for (int &x : data)    x = rng() % (N * 2);
    for (int &x : queries) x = rng() % (N * 2);

    // 1. Linear scan — ทดสอบกับข้อมูลน้อยกว่าแล้วประมาณ
    {
        const int S = 50000, SQ = 100;
        clock_t t = clock();
        volatile int found = 0;
        for (int i = 0; i < SQ; i++)
            for (int j = 0; j < S; j++)
                if (data[j] == queries[i]) { found++; break; }
        double e = (double)(clock()-t)/CLOCKS_PER_SEC;
        cout << "Linear scan   (N=" << S << ", Q=" << SQ << "): " << e << " sec\n";
        cout << "  ประมาณ (N=" << N << ", Q=" << Q << "): " << e*(N/S)*(Q/SQ) << " sec\n\n";
    }

    // 2. Binary search
    {
        vector<int> sd = data;
        sort(sd.begin(), sd.end());
        clock_t t = clock();
        volatile int found = 0;
        for (int i = 0; i < Q; i++)
            if (binary_search(sd.begin(), sd.end(), queries[i])) found++;
        double e = (double)(clock()-t)/CLOCKS_PER_SEC;
        cout << "Binary search (N=" << N << ", Q=" << Q << "): " << e << " sec\n\n";
    }

    // 3. unordered_set
    {
        unordered_set<int> uset(data.begin(), data.end());
        clock_t t = clock();
        volatile int found = 0;
        for (int i = 0; i < Q; i++)
            if (uset.count(queries[i])) found++;
        double e = (double)(clock()-t)/CLOCKS_PER_SEC;
        cout << "unordered_set (N=" << N << ", Q=" << Q << "): " << e << " sec\n\n";
    }

    // 4. unordered_set + reserve
    {
        unordered_set<int> uset;
        uset.reserve(N * 2);
        uset.max_load_factor(0.7);  // จำนวน bucket คำนวณจาก (N * 2) / max_load_factor
                                    // bucket ถูกใช้ได้สูงสุด 70% — ที่เหลืออีก 30% เป็น "พื้นที่กันชน"
        uset.insert(data.begin(), data.end());
        clock_t t = clock();
        volatile int found = 0;
        for (int i = 0; i < Q; i++)
            if (uset.count(queries[i])) found++;
        double e = (double)(clock()-t)/CLOCKS_PER_SEC;
        cout << "uset + reserve(N=" << N << ", Q=" << Q << "): " << e << " sec\n";
    }

    return 0;
}
```

12. เมื่อไหร่ที่ ไม่ควร ใช้ Hashing

สถานการณ์	โครงสร้างที่เหมาะกว่า	เหตุผล
ต้องการลำดับของ key	map / set	unordered_* ไม่เรียง
key เป็น int ช่วงเล็ก ≤ 10^6	array / vector	direct addressing เร็วกว่า
ต้องการ lower_bound / upper_bound	map / set	hash ไม่รองรับ
ต้องการ predecessor / successor	set	เดินใน BST ได้

ต้องการลำดับของ key → map / set
unordered_* เก็บ key ในลำดับสุ่มตาม hash ไม่เรียง ถ้าต้องการพิมพ์ key จากน้อยไปมาก ต้องใช้ set หรือ map ที่ใช้ BST ภายในซึ่งเรียงอัตโนมัติ

---

key เป็น int ช่วงเล็ก ≤ 10^6 → array / vector
ถ้า key เป็น int ตั้งแต่ 0–1,000,000 ไม่ต้อง hash เลย — ใช้ค่า key เป็น index array โดยตรงได้เลย

```
int freq[1000001] = {0};
freq[key]++;        // เข้าถึง O(1) เร็วที่สุด ไม่มี hash overhead
```

วิธีนี้เรียกว่า **direct addressing** — เร็วกว่า hash เพราะไม่ต้องคำนวณอะไรเพิ่ม

ต้องการ lower_bound / upper_bound → map / set
lower_bound(x) หมายถึง "หา key ที่น้อยที่สุดที่ ≥ x" — ต้องรู้ว่า key เรียงอยู่อย่างไร hash table ไม่รู้เรื่องลำดับจึงทำไม่ได้

```
set<int> s = {1, 3, 5, 7, 9};
s.lower_bound(4);  // → iterator ชี้ที่ 5 (น้อยที่สุดที่ ≥ 4)
```

ต้องการ predecessor / successor → set

- **predecessor** = ค่าที่ใหญ่ที่สุดที่ยังน้อยกว่า x
- **successor** = ค่าที่เล็กที่สุดที่ยังมากกว่า x

```
set<int> s = {1, 3, 5, 7, 9};
auto it = s.find(5);
--it;  // predecessor → 3
++it;  // กลับมาที่ 5
++it;  // successor   → 7
```

ทำได้เพราะ set ใช้ BST ที่เดิน iterator ไปซ้าย/ขวาได้ — hash table ไม่มีแนวคิดนี้ครับ

---

13. วิธีเลือกโครงสร้างอย่างรวดเร็ว

```
ต้องการเช็ค membership เร็ว?          → unordered_set
ต้องการนับความถี่?                    → unordered_map
ต้องการลำดับ / lower_bound?           → set / map
key เป็น int ช่วงเล็ก (≤ 10^6)?       → int freq[MAXVAL]
key เป็น pair/struct?                 → unordered_map + custom hash
```

---

14. 🏋 แบบฝึกหัด

ข้อ 1 — Distinct Values

โจทย์: รับตัวเลข N ตัว ให้พิมพ์จำนวนค่าที่แตกต่างกัน โดยนับค่าเดียวกันเป็น 1 ค่าเสมอ
Hint: ใส่ครบแล้ว uset.size() คือคำตอบ

ข้อ 2 — Most Frequent Word

โจทย์: รับคำ N คำ ให้พิมพ์คำที่ปรากฏบ่อยที่สุด
Hint: unordered_map<string, int> นับความถี่ แล้ว iterate หาค่าสูงสุด

ข้อ 3 — Two Sum

โจทย์: รับตัวเลข N ตัว และเป้าหมาย T ให้หาว่ามีคู่บวกกันได้ T ไหม
Hint: สำหรับแต่ละ x เช็คว่ามี (T-x) อยู่ใน map แล้วไหม — O(N) ทั้งหมด

สรุป

```
Manual hash table  → เข้าใจกลไก แต่เขียนเองยุ่งยาก
STL unordered_*    → ใช้งานง่าย O(1) เฉลี่ย เหมาะกับ contest
```

3 Use Case หลักในการแข่งขัน:

- เช็ค membership: unordered_set
- นับความถี่: unordered_map
- mapping key → value: unordered_map